9.3 3D CNNs (C3D)

- in 2D CNNs, 2D convolution is performed at the convolutional layers to extract features from local neighborhood on feature maps in the previous layer
- then an additive bias is applied and the result is passed through an activation function
- formally, the value of an unit at position (x, y) in the jth feature map in the ith layer, denoted as v_{ij}^{xy}, is given by

$$v_{ij}^{xy} = \phi\left(b_{ij} + \sum_{m}\sum_{p=0}^{P_i-1}\sum_{q=0}^{Q_i-1} w_{ijm}^{pq} v_{(i-1)m}^{(x+p)(y+q)}\right),$$

  where ϕ is the activation function, b_{ij} is the bias for this feature map, m indexes over the set of feature maps in the $(i-1)$th layer connected to the current feature map, w_{ijk}^{pq} is the value at the position (p, q) of the kernel connected to the kth feature map, and P_i and Q_i are the height and width of the kernel, respectively
- in the downsampling layers, the resolution of the feature maps is reduced by pooling over local neighborhood on the feature maps in the previous layer, thereby enhancing the invariance to distortions on the inputs
- a CNN architecture can be constructed by stacking multiple layers of convolution and downsampling in an alternating fashion

9.3 3D CNNs (C3D)

- in 2D CNNs, convolutions are applied on the 2D feature maps to compute features from the spatial dimensions only
- when applied to video analysis problems, it is desirable to capture the motion information encoded in multiple contiguous frames
- to this end, 3D convolutions can be used in the convolution stages of CNNs to compute features from both spatial and temporal dimensions
- the 3D convolution is achieved by convolving a 3D kernel to the cube formed by stacking multiple contiguous frames together
- by this construction, the feature maps in the convolution layer are connected to multiple contiguous frames in the previous layer, thereby capturing motion information
- formally, the value at position (x, y, z) on the jth feature map in the ith layer is given by

$$v_{ij}^{xyz} = \phi\left(b_{ij} + \sum_{m}\sum_{p=0}^{P_i-1}\sum_{q=0}^{Q_i-1}\sum_{r=0}^{R_i-1} w_{ijm}^{pqr}\, v_{(i-1)m}^{(x+p)(y+q)(z+r)}\right),$$

  where R_i is the size of the 3D kernel along the temporal dimension, w_{ijm}^{pqr} is the (p, q, r)th value of the kernel connected to the mth feature map in the previous layer
- a comparison of 2D and 3D convolutions is given in Figure 6

9.3 3D CNNs (C3D)

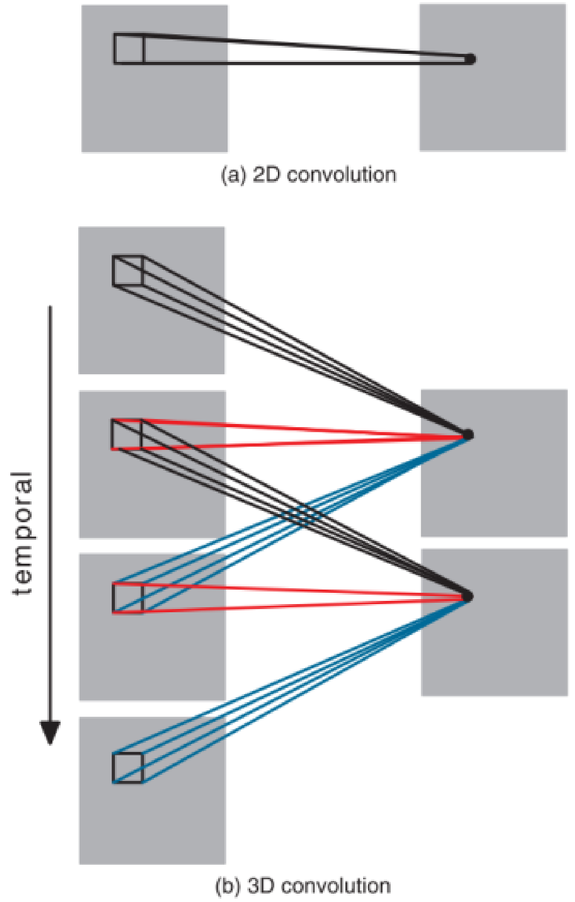


Figure 6: Comparison of (a) 2D and (b) 3D convolutions. In (b) the size of the convolution kernel in the temporal dimension is 3 and the sets of connections are color-coded so that the shared weights are in the same color. In 3D convolution, the same 3D kernel is applied to overlapping 3D cubes in the input video to extract motion features.

9.3 3D CNNs (C3D)

- note that a 3D convolutional kernel can only extract one type of features from the frame cube, since the kernel weights are replicated across the entire cube
- a general design principle of CNNs is that the number of feature maps should be increased in late layers by generating multiple types of features from the same set of lower-level feature maps
- similarly to the case of 2D convolution, this can be achieved by applying multiple 3D convolutions with distinct kernels to the same location in the previous layer (Figure 7)

9.3 3D CNNs (C3D)

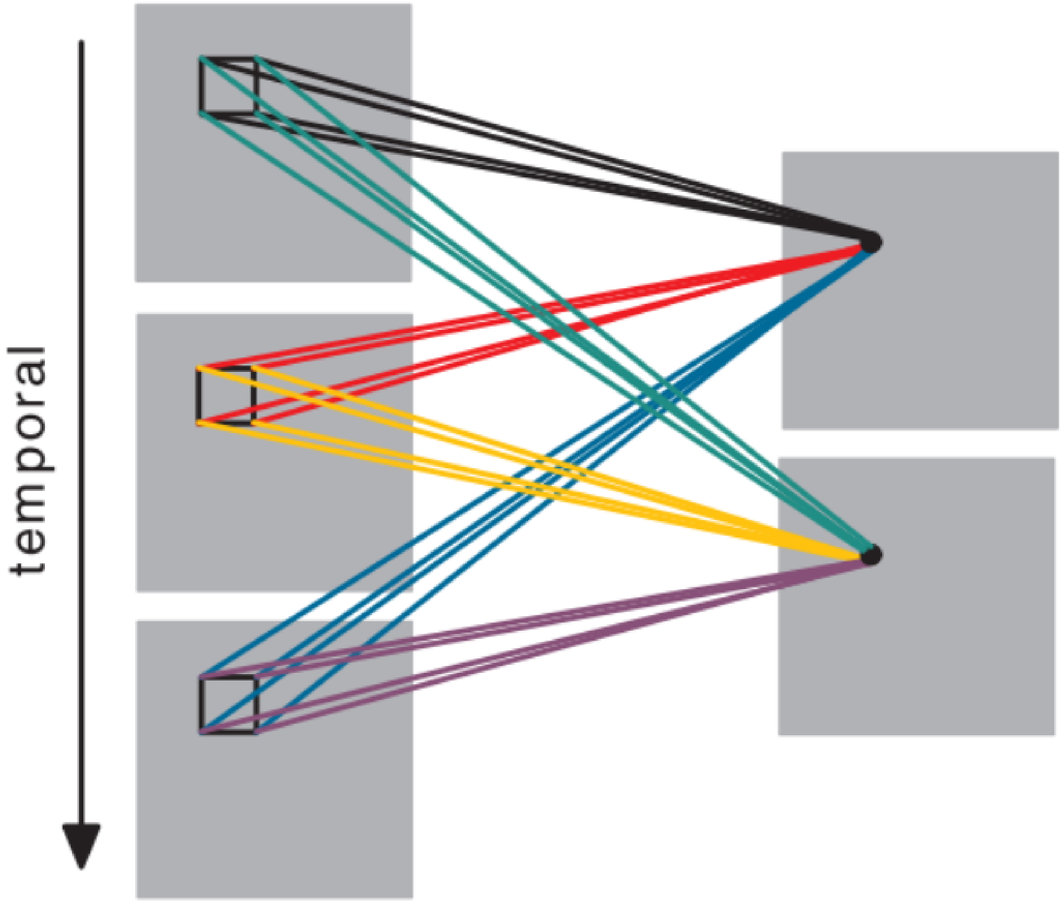


Figure 7: Extraction of multiple features from contiguous frames. Multiple 3D convolutions can be applied to contiguous frames to extract multiple features. As in Figure 6, the sets of connections are color-coded so that the shared weights are in the same color. Note that all six sets of connections do not share weights, resulting in two different feature maps on the right.

9.3 3D CNNs (C3D)

- now, we explain in detail the basic operations of 3D CNNs and analyze different architectures for 3D CNNs
- 3D CNNs are believed to be well-suited for spatio-temporal feature learning
- compared to 2D CNNs, 3D CNNs have the ability to model temporal information better, owing to 3D convolution and 3D pooling operations
- in 3D CNNs, convolution and pooling operations are performed spatio-temporally, while in 2D CNNs they are done only spatially
- Figure 6 illustrates the difference: 2D convolution applied on an image will output an image, 2D convolution applied on multiple images (treating them as different channels) also results in an image
- hence, 2D CNNs lose temporal information of the input signal right after every convolution operation
- only 3D convolution preserves the temporal information of the input signals, resulting in an output volume
- the same phenomena is applicable for 2D and 3D polling

9.3 3D CNNs (C3D)

- now, we try to identify a good architecture for 3D CNNs
- according to the findings in 2D CNNs, small receptive fields of 3×3 convolution kernels with deeper architectures yield best results
- hence, for the architecture search study, the spatial receptive field is fixed to 3×3, and only the temporal depth of the 3D convolution kernels is varied
- **Notations**: for simplicity, from now on, we refer to video clips with a size of $c \times l \times h \times w$, where c is the number of channels, l is length in number of frames, h and w are the height and width of the frame, respectively
- we also refer to 3D convolution and pooling kernel size by $d \times k \times k$, where d is the kernel temporal depth and k is the kernel spatial size

9.3 3D CNNs (C3D)

- **Common network settings**: we now describe the network settings that are common to all the networks
- the networks are set up to take video clips as inputs and predict the class labels
- all video frames are resized into 128×171
- videos are split into non-overlapped 16-frame clips, which are then used as input to the networks
- the input dimensions are $3 \times 16 \times 128 \times 171$
- the networks have 5 convolution layers and 5 pooling layers (each convolution layer is immediately followed by a pooling layer), 2 fully-connected layers, and a softmax loss layer to predict action labels

9.3 3D CNNs (C3D)

- the number of filters for the 5 convolution layers from 1 to 5 are 64, 128, 256, 256, 256, respectively
- all convolution kernels have a size of d, where d is the kernel temporal depth (we will later vary the value d of these layers to search for a good 3D architecture)
- all of these convolution layers are applied with appropriate padding (both spatial and temporal) and stride 1, thus there is no change in terms of size from the input to the output of these convolution layers
- all pooling layers are max-pooling with kernel size $2 \times 2 \times 2$ (except for the first layer) with stride 1, which means the size of output signal is reduced by a factor of 8 compared with the input signal
- the first pooling layer has kernel size $1 \times 2 \times 2$ with the intention of not to merge the temporal signal too early and also to satisfy the clip length of 16 frames (e.g., we can temporally pool with factor 2 at most 4 times before completely collapsing the temporal signal)
- the two fully-connected layers have 2048 outputs

9.3 3D CNNs (C3D)

- **Varying network architectures**: we are mainly interested in how to aggregate temporal information through the deep networks
- to search for a good 3D CNN architecture, only the kernel temporal depth d_i of the convolution layers is varied, while keeping all other common settings fixed as stated above
- two types of architectures are experimented with: 1) homogeneous temporal depth: all convolution layers have the same kernel temporal depth; and 2) varying temporal depth: kernel temporal depth is changing across the layers
- for homogeneous setting, 4 networks having kernel temporal depth of d equal to 1, 3, 5, and 7 are experimented with
- these networks are named as *depth-d*, where d is their homogeneous temporal depth
- note that *depth-1* net is equivalent to applying 2D convolutions on separate frames
- for the varying temporal depth setting, two networks with temporal depth increasing: 3-3-5-5-7 and decreasing: 7-5-5-3-3 from the first to the fifth convolution layer, respectively, are experimented with

9.3 3D CNNs (C3D)

- note that all of these networks have the same size of the output signal at the last pooling layer, thus they have the same number of parameters for the fully-connected layers
- their number of parameters is only different at convolution layers due to different kernel temporal depth
- these differences are quite minor compared to millions of parameters in the fully-connected layers
- for example, any two of the above networks with temporal depth difference of 2, only have 17K parameters fewer or more from each other
- the biggest difference in number of parameters is between *depth-1* network and *depth-7* network, where *depth-7* network has 51K more parameters, which is less than 0.3% of the total of 17.5 million parameters of each network
- this indicates that the learning capacities of the networks are comparable and the differences in number of parameters should not affect the results of the architecture search

9.3 3D CNNs (C3D)

- it is found that *depth-3* performs best among the homogeneous networks
- note that *depth-1* is significantly worse than the other networks, which is believed to be due to lack of motion modeling
- compared to the varying temporal depth networks, *depth-3* is the best performer, but the gap is smaller
- this suggests $3 \times 3 \times 3$ is the best kernel choice for 3D CNNs and 3D CNNs are consistently better than 2D CNNs for video classification

9.3 3D CNNs (C3D)

- **Network architecture**: the architecture search findings indicate that the homogeneous setting with convolution kernels of $3 \times 3 \times 3$ is the best option for 3D CNNs
- this finding is also consistent with a similar finding in 2D CNNs
- with a large-scale dataset, one can train a 3D CNN with $3 \times 3 \times 3$ kernel as deep as possible, subject to the machine memory limit and computation affordability
- the proposed 3D CNN is designed to have 8 convolution layers, 5 pooling layers, followed by two fully-connected layers, and a softmax output layer
- the network architecture is presented in Figure 8
- this network is called C3D
- all of 3D convolution filters are $3 \times 3 \times 3$ with stride $1 \times 1 \times 1$
- all 3D pooling layers are $2 \times 2 \times 2$ with stride $2 \times 2 \times 2$, except for `pool1`, which has kernel size of $1 \times 2 \times 2$ and stride $1 \times 2 \times 2$, with the intention of preserving the temporal information in the early phase
- each fully-connected layer has 4096 output units

9.3 3D CNNs (C3D)

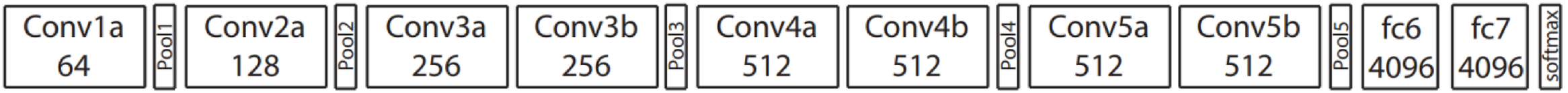


Figure 8: C3D architecture. C3D net has 8 convolution, 5 max-pooling, and 2 fully-connected layers, followed by a softmax output layer. All 3D convolution kernels are $3 \times 3 \times 3$ with stride 1 in both spatial and temporal dimensions. Number of filters are denoted in each box. The 3D pooling layers are denoted from `pool1` to `pool5`. All pooling kernels are $2 \times 2 \times 2$, except for `pool1`, which is $1 \times 2 \times 2$. Each fully-connected layer has 4096 output units.